Explainable Deep Reinforcement Learning: How Does AI Play Atari Games?

Thomas Vroom (i6291496)

Abstract

Deep reinforcement learning has achieved great results in a variety of domains, but it suffers from a significant loss in interpretability that raises concerns regarding its deployment in critical areas. Explainable reinforcement learning has emerged as a field aiming to develop techniques that provide insights into the decision-making of deep reinforcement learning agents. In this literature review we critically analyze the work of five state-of-the-art papers in explainable reinforcement learning, ranging from hand-crafted saliency maps to the learning of inherently interpretable surrogate models. We select and compare these works on the shared domain of Atari game-playing, one of the most popular deep learning benchmarks. Our findings reveal that high quality, layman-interpretable explanations of deep reinforcement learning agents are possible, but at the cost of scalability and freedom of policy architecture. Saliency maps prove a useful tool for visualizing feature importance, but are susceptible to noise which limits their effectiveness as automated explanations.

ACM Reference Format:
Thomas Vroom (i6291496). 2026. Explainable Deep Reinforcement Learning: How Does AI Play Atari Games?. In . ACM, New York, NY, USA, 5 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

1 Introduction

In The Challenge of Reinforcement Learning[1], Sutton states:

> Part of the appeal of reinforcement learning is that it is in a sense the whole AI problem in a microcosm.

Reinforcement Learning (RL) concerns the problem of an autonomous agent learning how to interact with its environment to achieve a goal, a problem that lies at the heart of the study of artificial intelligence[1, 2]. When combined with deep learning, RL agents have achieved breakthrough performances in game-playing, robotics, natural language processing, and many more domains[3].

Deep Reinforcement Learning (DRL) is however not without its limitations; DRL agents are often seen as 'black-box' systems due to the complexity of their internal architecture, where millions of interactions between parameters render their decision-making incomprehensible to most human observers[3, 4]. This lack of interpretability raises concerns regarding the deployment of DRL agents in critical areas such as autonomous vehicles and healthcare services, and limits the degree to which humans can learn from strategies developed by these agents[4, 5]. To address these concerns, Explainable Reinforcement Learning (XRL) has emerged as a field aiming to develop techniques that provide insights into the decision-making process of DRL agents[5].

In this literature review, we offer a critical analysis and comparison of five state-of-the-art methods for explaining the decision-making of DRL agents. To aid our comparison, we select our methods based on a shared domain: The playing of Atari games, one of the most popular benchmarks for DRL agents[6].

The rest of this paper is structured as follows: First we will discuss relevant background information on RL and other related topics in Section 2. We will summarize, compare, and critically analyze a selection of five state-of-the-art methods in Section 3. Finally, we will draw our conclusions and pose directions for future research in Section 4.

2 Background

We will first discuss some relevant background information.

2.1 Reinforcement Learning

Reinforcement Learning is a major paradigm of machine learning, focused on autonomous agents that learn to achieve a goal through interacting with their environment[1, 5, 7]. RL agents are trained on sequential decision-making problems where the environment can be modeled as a *Markov Decision Process* (MDP), specified by the tuple $(\mathcal{S}, \mathcal{A}, P, R, \gamma)$: [5, 7]

- $\mathcal{S}$: The set of states representing possible configurations of the environment.
- $\mathcal{A}$: The set of possible actions available to the agent.
- $P(s'|s,a)$: The transition probability function that describes the probability of transitioning to state s' after performing action a in state s.
- $R(s,a)$: The immediate reward obtained after performing action a in state s.
- $\gamma \in (0,1)$: The *discount factor*, balancing the importance of immediate and future rewards.

Reinforcement Learning is the process of learning a *policy*, a function $\pi(s,a) : \mathcal{S} \rightarrow \mathcal{A}$ that defines which action to take in each state, solely through interacting with the environment (in the context of this paper, the word *policy* is used interchangeably with the word *strategy*). The goal is to find an optimal policy $\pi^\star$ that maximizes the total (expected) reward[5]. What makes learning an optimal policy challenging is the inherent trade-off between *exploration* and *exploitation*: The agent has to exploit its learned knowledge to obtain reward, but to learn said knowledge it must explore other actions as well[7].

The *value function* of an agent is a function $V^\pi(s) : \mathcal{S} \rightarrow \mathbb{R}$ that describes the long-term reward the agent expects after visiting a

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference'17, Washington, DC, USA
© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

state s, formally defined as: [5, 7]

$$V^{\pi}(s) = \sum_{a \in \mathcal{A}} \pi(a|s) \left[R(s,a) + \gamma \sum_{s' \in \mathcal{S}} P(s'|s,a) V^{\pi}(s') \right] \quad (1)$$

The value function is continually updated by the agent based on the experienced consequences of its actions, and often forms the foundation of an agent's decision making[7].

A simple, but effective architecture for RL is the *Actor-Critic* framework. This framework consists of an *actor* who interacts with the environment, proposing what action to take in the current state, and a *critic* who evaluates the actions taken by the actor[8]. In its most common form, the critic is a value function that gets updated based on the rewards obtained by the actor; The actor is a greedy policy that selects the action that leads to the highest (expected) long-term reward, according to the value function. By letting this process run for an infinite amount of time, it is guaranteed to converge to an optimal policy[8].

Q-Learning is possibly the most well-known RL algorithm, being applicable to environments even without explicit knowledge of the underlying MDP, while still maintaining strong convergence guarantees[9]. During Q-learning the agent estimates a *Q-value* $Q(s,a)$, defined as the expected long-term reward of performing action a in state s, then greedily selects the action with the highest Q-value in the current state[5, 9].

2.2 Deep Reinforcement Learning

While Q-learning has been proven to be very effective in small, structured environments, it suffers greatly from the curse of dimensionality[5]. To address this limitation, DRL agents use neural networks or other function approximators to replace the explicit enumeration of policies or value functions[5, 10].

A simple, but effective example of DRL is *Deep Q-Networks* (DQNs), utilizing a deep neural network to approximate optimal Q-values[6, 10]. This network takes in a state input s, processes this input through several layers of a neural network, then predicts the Q-values for all possible actions $\mathcal{A}$ simultaneously[5].

One of the most popular benchmarks for DRL agents is the playing of *Atari games*. This started to gain popularity after the release of the Arcade Learning Environment (ALE), an emulator of the Atari 2600 game console that supports over 50 different Atari games[6, 11]. While these games may appear simple in concept, they provide high-dimensional sensory input data through RGB images at 60 Hz. Additionally, the majority of these games were designed to be difficult and engaging for human players, making them an excellent benchmark for (artificial) intelligence[6, 10]. The introduction of (convolutional) DQNs caused a major breakthrough in the playing of Atari games, paving the way for superhuman performance[10].

2.3 Explainable Reinforcement Learning

The main concern regarding DRL agents and deep learning in general is the loss of interpretability, with the complex architecture of a neural network being near incomprehensible for most human observers[3, 4]. The field of XRL aims to develop techniques that provide insight into the decision-making of these agents, and can generate explanations for non-expert observers[5]. The authors of [5] define four categories of techniques for explaining DRL agents:

- *Feature-level Explanation Methods*: Methods that aim to identify the most important features in the agent's observation space that influence its decision-making.
- *State-level Explanation Methods*: Methods that focus on identifying critical states in the agent's trajectory that significantly impact its performance.
- *Dataset-level Explanation Methods*: Methods that focus on understanding how specific training examples influence the learned policy of the agent.
- *Model-level Explanation Methods*: Methods that focus on the self-explainability of policies, aiming to make the agent's decision-making process inherently interpretable.

Since we are mainly concerned with the interpretability loss of DRL agents, we will focus on *feature-level* and *model-level* explanation methods in this review.

Because Atari games have a high-dimensional, visual input space, a useful tool for explaining feature importance is *Saliency Maps*: visual heatmaps that highlight the most important regions influencing the agent's behavior, often constructed through the accumulation of gradients[12].

3 Literature Review

This section outlines our methodology for selecting papers, followed by an overview and critical analysis of the selected works. A concise overview of the main points discussed in this section can be found in Table 1.

3.1 Methodology

We searched for candidate papers on Google Scholar with the following keywords: 'Deep Reinforcement Learning', 'Explainable', and 'Atari'. After sorting the results by relevance, we went through the first three pages and filtered out any papers that did not meet the following inclusion criteria:

- The paper introduces at least one novel method for generating explanations (no surveys).
- The paper comes from a peer-reviewed journal or conference.
- The work is evaluated on Atari games.
- The generated explanations are *feature-level* or *model-level*.

This process gave us a selection of eight papers: [13–20]. After additional reading we decided to omit [18] and [19] from this review, as they both use minor variations of the well-established Grad-CAM algorithm[21] to generate saliency maps, which is already well-represented in the remainder of our selection. We also decided to omit [20] as they use a very similar approach to [16], with the latter being more in-depth. This gave us a final selection of five papers: [13–17].

3.2 Overview

We will now give a short overview of the selected methods in order of publication date:

[13]. The authors of this paper train and evaluate a DQN while recording the activations of the last hidden layer and some auxiliary hand-crafted features (Q-value estimates, rewards, played actions,

raw frames, etc.) for each visited state. They then apply the t-SNE dimensionality reduction algorithm[22] on the activations data and visualize the results while allowing coloring and filtering of data points based on the auxiliary features. The final analysis and generation of explanations is done manually.

[14]. The authors of this paper introduce a method to generate perturbation-based saliency maps of an agent's policy and value function. This method consists of adding spatial uncertainty, e.g. a Gaussian blur, to a certain part of the input space, followed by measuring how much the policy or value estimation changes in response. This gets repeated for every pixel of an input frame to generate a full saliency map.

[15]. The authors of this paper introduce an adjustment to the layer-wise relevance propagation (LRP) z^+ rule[23] for generating saliency maps. They first adjust the rule to work with DQNs, then introduce an argmax operator in the convolutional layers to focus on the most contributing neurons. According to the authors, this makes the saliency maps more selective and easier to interpret for people who are not domain experts.

[16]. The authors of this paper introduce a novel policy architecture that generates saliency maps as part of its forward pass procedure, requiring no further post-hoc analysis. The saliency maps are generated in real-time using a soft attention module that is applied at the end of the feature extraction process. The authors show that, despite this unusual policy architecture, the performance cost remains minimal.

[17]. The authors of this paper first extract objects of interest from the input space, and use their characteristics like position and velocity to create a more percept-like input representation. They then train an inherently interpretable surrogate model (e.g. a decision tree) to imitate the behavior of the original policy. This method is the only *model-level* explanation method of our selection.

3.3 Analysis

A unique property of [13] and [17] is that these are the only two methods in our selection that are able to identify *hierarchical policies*: Policies that break down complex objectives into several sub-goals, changing their strategy depending on what sub-goal they are currently focusing on. This is especially relevant in the context of Atari games, as the strategies of expert human players can often be expressed as hierarchical policies as well[24].

The standout feature of [13] is that the authors are able to generate very detailed, case-specific, layman-interpretable explanations. A big contributor to this is the manual analysis component, which is also its biggest limitation: Not only is a domain expert required to analyze the final visualization and provide explanations, the features shown in the visualization need to be hand-crafted for each game as well. This is of course a huge limitation in terms of scalability, but also restricts the quality of the explanations to human interpretation.

Having humans generate explanations of deep learning models adds an inherent bias to human-interpretable strategies, and makes it more challenging to identify issues like model overfitting. An example of this in the game of *Pong* is discussed in [14]:

> Our initial understanding of the kill shot ... was that the agent had learned to first 'lure' the opponent into the lower region of the frame and then aim the ball towards the top of the frame, where it was difficult to return. Saliency visualizations told a different story ... we see that the agent attends to very little besides its own paddle: not even the ball.

Saliency maps, as used by [14–16], suffer less from this issue as they directly visualize what the policy (seemingly) attends to. However, this comes with the cost of potentially being less interpretable for non-experts, depending on how the saliency map is generated.

Whereas saliency maps are most frequently constructed using gradient-based methods, the authors of [14] opt for a perturbation approach to make the saliency maps less noisy and more interpretable. In the context of reinforcement learning, perturbation can be used to generate saliency maps for both the policy as well as the value function of an agent, allowing for explanations of both the actor and the critic in actor-critic methods.

As discussed in [25], the main limitation of constructing saliency maps using perturbation is that the map only shows how sensitive the agent is to that specific perturbation method. It can thus be questioned whether these saliency maps actually explain the decision making of the agent, or simply show which parts of the input space are most sensitive to spatial uncertainty. In addition to this, the method introduced in [14] applies the perturbation procedure pixel-wise on the input space, adding a substantial computational overhead and limiting scalability.

To prevent excessive computational overhead, the authors of [15] generate saliency maps using a different technique called layer-wise relevance propagation (LRP). This technique is a lot more efficient than perturbation-based methods, requiring only a single backward pass through the network to assign *relevance scores* to neurons.

To make the saliency maps more selective, the authors of [15] adjust the commonly used z^+ rule by including an argmax operator at the end of every layer, focusing on only the most important neurons. While this adjustment does remove a lot of noise from the saliency maps, it comes with the risk of being *too* selective, hiding relevant information.

In addition to this, it should be noted that this method for constructing saliency maps is only compatible with policies that use a DQN architecture. While this is a very common and useful policy architecture, it is more restrictive than the work of [13, 14], which is compatible with all[1] policy architectures.

A common factor between the methods introduced by [13–15] is that they all provide *post-hoc* explanations, analyzing and interpreting the decision-making process of an agent after it has made predictions[26]. The authors of [16] introduce a highly-specific policy architecture that can construct saliency maps in real time.

Since this policy architecture is able to reach a performance comparable to state-of-the-art baselines in their experiments, the authors claim that the simultaneous construction of saliency maps is a 'free lunch' process. While these performance claims are true, we have to note that the constructed saliency maps are of noticeably lower quality than those constructed by the works of [14, 15] among others.

Table 1: Comparison of selected methods on various criteria.

Paper	Method	Scalability	Timing	Policy	Strengths	Limitations
[13]	t-SNE with manual analysis	Very low	Post-hoc	Any[1]	+ case-specific and detailed explanations + can identify hierarchical policies	- expert analysis and feature-crafting required - human interpretation adds bias
[14]	Perturbation-based saliency maps	Low	Post-hoc	Any[1]	+ less noisy saliency maps + saliency maps for actor-critic methods	- high computational cost - only measures policy's sensitivity to perturbation
[15]	LRP-based saliency maps	High	Post-hoc	Restricted	+ less noisy saliency maps + low computational cost	- can be *too* selective - only works with DQNs
[16]	Saliency maps directly from policy	High	Real-time	Restricted	+ real-time saliency maps, no performance cost + low computational cost	- noisy saliency maps - requires specific policy architecture
[17]	Inherently interpretable surrogate model	Low	Ante-hoc/ Post-hoc[2]	Restricted	+ global explanations + can identify hierarchical policies	- unreliable feature encoding - model is surrogate

To address this issue, the authors also introduce a second policy architecture that is more tailored to the generation of higher quality saliency maps. However, this architecture in turn suffers from a noticeable performance deficit compared to the same baseline methods.

To round off our selection of methods, the authors of [17] train an inherently interpretable surrogate model to replicate the decision making of a DRL agent's original 'black-box' policy. What sets this approach apart from the other works is that an inherently interpretable model provides model-level, *ante-hoc* explanations, assuming that the features used by the model are human-interpretable[26].

Since the input space of Atari games is purely pixel-based, the authors introduce an automatic feature encoding algorithm that efficiently identifies objects of interest from a single frame. While the automation of this process is great for scalability, the authors admit that the method itself is not particularly reliable, achieving mixed results in more complex games and even outright failing in others.

In addition to this, we should also note that the inherently interpretable model is still an approximation of the original policy, and therefore not guaranteed to mimic the agent's decision making exactly. This comes alongside the risk of overfitting on the original policy, and the computational overhead of having to train two models instead of one.

4 Conclusion

In this literature review we highlight the importance of interpretability in deep reinforcement learning, the lack of which has been a cause of concern in critical domains and limits the degree to which humans can learn from DRL agents. We summarized, compared, and critically analyzed five state-of-the-art methods for explaining the decision-making process of a DRL agent, a concise overview of which can be found in Table 1.

Our analysis reveals that high quality, layman-interpretable explanations of DRL agents are possible, but at the cost of scalability or freedom of architecture. The best explanations are generated by methods that still involve human experts in the loop, with fully automated pipelines not yet reaching the same consistent quality. Saliency maps prove a useful tool for explaining what elements of the input space the agent attends to, but they are susceptible to noise which limits their effectiveness as explanations for non-experts.

We identify multiple possible directions for future research: Both the human-in-the-loop as well as the automated methods could learn from each other's strengths to work towards fully automated, high quality explanations. There is also room to further improve the quality of saliency maps, possibly through attention-based architectures like transformers that have already seen success in other vision-based applications. Lastly, user studies should be performed to get an accurate understanding of the gap between layman- and expert-interpretability of the generated explanations, helping to get a clearer picture of the gap between advancements in XRL and the applicability of DRL agents in the real world.

[1] Compatible with any policy architecture, as long as access to logits is provided.

[2] Although the surrogate model itself is an *ante-hoc* method, it needs to be trained on traces of an existing policy, and can thus also be seen as a *post-hoc* method.

References

[1] Richard S Sutton. Introduction: The challenge of reinforcement learning. In *Reinforcement learning*, pages 1–3. Springer, 1992.

[2] David Abel, André Barreto, Benjamin Van Roy, Doina Precup, Hado P van Hasselt, and Satinder Singh. A definition of continual reinforcement learning. *Advances in Neural Information Processing Systems*, 36:50377–50407, 2023.

[3] Yuxi Li. Deep reinforcement learning: An overview, 2018.

[4] Emrullah ŞAHiN, Naciye Nur Arslan, and Durmuş Özdemir. Unlocking the black box: an in-depth review on interpretability, explainability, and reliability in deep learning. *Neural computing and applications*, 37(2):859–965, 2025.

[5] Zelei Cheng, Jiahao Yu, and Xinyu Xing. A survey on explainable deep reinforcement learning. *arXiv preprint arXiv:2502.06869*, 2025.

[6] Ionel-Alexandru Hosu and Traian Rebedea. Playing atari games with deep reinforcement learning and human checkpoint replay, 2016.
[7] Barto Andrew and Sutton Richard S. *Reinforcement learning: an introduction*. The MIT Press, 2018.
[8] Vijay Konda and John Tsitsiklis. Actor-critic algorithms. *Advances in neural information processing systems*, 12, 1999.
[9] Christopher JCH Watkins and Peter Dayan. Q-learning. *Machine learning*, 8(3):279–292, 1992.
[10] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *nature*, 518(7540):529–533, 2015.
[11] Marc G Bellemare, Yavar Naddaf, Joel Veness, and Michael Bowling. The arcade learning environment: An evaluation platform for general agents. *Journal of artificial intelligence research*, 47:253–279, 2013.
[12] Karen Simonyan, Andrea Vedaldi, and Andrew Zisserman. Deep inside convolutional networks: Visualising image classification models and saliency maps. *arXiv preprint arXiv:1312.6034*, 2013.
[13] Tom Zahavy, Nir Ben-Zrihem, and Shie Mannor. Graying the black box: Understanding dqns, 2016.
[14] Samuel Greydanus, Anurag Koul, Jonathan Dodge, and Alan Fern. Visualizing and understanding atari agents. In *International conference on machine learning*, pages 1792–1801. PMLR, 2018.
[15] Tobias Huber, Dominik Schiller, and Elisabeth André. Enhancing explainability of deep reinforcement learning through selective layer-wise relevance propagation. In *Joint German/Austrian Conference on Artificial Intelligence (Künstliche Intelligenz)*, pages 188–202. Springer, 2019.
[16] Dmitry Nikulin, Anastasia Ianina, Vladimir Aliev, and Sergey Nikolenko. Free-lunch saliency via attention in atari agents. In *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*, pages 4240–4249. IEEE, 2019.
[17] Alexander Sieusahai and Matthew Guzdial. Explaining deep reinforcement learning agents in the atari domain through a surrogate model. In *Proceedings of the AAAI Conference on Artificial Intelligence and Interactive Digital Entertainment*, volume 17, pages 82–90, 2021.
[18] Laurens Weitkamp, Elise van der Pol, and Zeynep Akata. Visual rationalizations in deep reinforcement learning for atari games. In *Benelux conference on artificial intelligence*, pages 151–165. Springer, 2018.
[19] Ho-Taek Joo and Kyung-Joong Kim. Visualization of deep reinforcement learning using grad-cam: how ai plays atari games? In *2019 IEEE conference on games (CoG)*, pages 1–2. IEEE, 2019.
[20] Alexander Mott, Daniel Zoran, Mike Chrzanowski, Daan Wierstra, and Danilo Jimenez Rezende. Towards interpretable reinforcement learning using attention augmented agents. *Advances in neural information processing systems*, 32, 2019.
[21] Ramprasaath R Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-cam: Visual explanations from deep networks via gradient-based localization. In *Proceedings of the IEEE international conference on computer vision*, pages 618–626, 2017.
[22] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.
[23] Sebastian Bach, Alexander Binder, Grégoire Montavon, Frederick Klauschen, Klaus-Robert Müller, and Wojciech Samek. On pixel-wise explanations for non-linear classifier decisions by layer-wise relevance propagation. *PloS one*, 10(7):e0130140, 2015.
[24] Hua Huang and Adrian Barbu. Playing atari ball games with hierarchical reinforcement learning, 2019.
[25] Tobias Huber, Benedikt Limmer, and Elisabeth André. Benchmarking perturbation-based saliency maps for explaining atari agents, 2022.
[26] Carl O. Retzlaff, Alessa Angerschmid, Anna Saranti, David Schneeberger, Richard Röttger, Heimo Müller, and Andreas Holzinger. Post-hoc vs ante-hoc explanations: xai design guidelines for data scientists. *Cognitive Systems Research*, 86:101243, 2024.